

CODSOFT INTERNSHIP – Tic-Tac-Toe

Project No: 2- Tic-Tac-Toe

```
def check_winner(board):  
    win_positions = [  
        [0,1,2],[3,4,5],[6,7,8], # rows  
        [0,3,6],[1,4,7],[2,5,8], # cols  
        [0,4,8],[2,4,6]          # diagonals  
    ]  
    for pos in win_positions:  
        if board[pos[0]] == board[pos[1]] == board[pos[2]] != '':  
            return board[pos[0]]  
    return None
```

Explanation:

Step 1: What is win_positions?

These are all possible ways to win in Tic-Tac-Toe

There are exactly 8 winning combinations:

- 3 rows
- 3 columns
- 2 diagonals

Example:

[0,1,2] → top row

[3,4,5] → middle row

[6,7,8] → bottom row

[0,3,6] → first column

[1,4,7] → second column

[0,4,8] → diagonal

[2,4,6] → other diagonal

So instead of writing many if statements, you store all cases in one list (very smart design).

Step 2: Loop Through Each Winning Case

for pos in win_positions:

You are checking each possible way to win one by one

Step 3: The Most Important Line

if board[pos[0]] == board[pos[1]] == board[pos[2]] != '':

Let's break this:

Part 1:

board[pos[0]] == board[pos[1]] == board[pos[2]]

Means:

- **All 3 positions have same value**

Example:

['X', 'X', 'X'] #Right

['O', 'O', 'O'] #Right

['X', 'O', 'X'] #Wrong

Part 2:

!= ''

Ensures:

- **It's not empty space**

Because:

[' ', ' ', ' '] (not a win)

Combined Meaning:

This line checks:

“Are all three positions same AND not empty?”

If yes → we found a winner

Step 4: Return Winner

return board[pos[0]]

Why pos[0]?

Because:

- All 3 are same
- So returning any one gives the winner

Example:

['X','X','X'] → return 'X'

Step 5: If No Winner Found

return None

Means:

- No one has won yet
 - Game continues
-

Full Intuition (VERY IMPORTANT)

Think like AI:

“Check every possible winning line”

“If any line has same symbols → winner found”

Visual Understanding

Board:

```
X | X | X
O | O |
| |
```

Loop checks:

- `[0,1,2] → X,X,X → return 'X'`
-

Why This Approach is Powerful

- ✓ Clean code
 - ✓ No repetition
 - ✓ Scalable logic
 - ✓ Easy to use in Minimax
-

Final One-Line Understanding

This function:

“Scans all 8 winning patterns and returns the player who completes any of them.”